

使用Spring Boot Admin实现运维监控平台

官方地址: <https://github.com/codecentric/spring-boot-admin>

我们知道, 使用Actuator可以收集应用系统的健康状态、内存、线程、堆栈、配置等信息, 比较全面地监控了Spring Boot应用的整个生命周期。但是还有一个问题: 如何呈现这些采集到的应用监控数据、性能数据呢? 在这样的背景下, 就诞生了另一个开源软件Spring Boot Admin。下面就来介绍什么是Spring Boot Admin以及如何使用Spring Boot Admin搭建完整的运维监控平台。

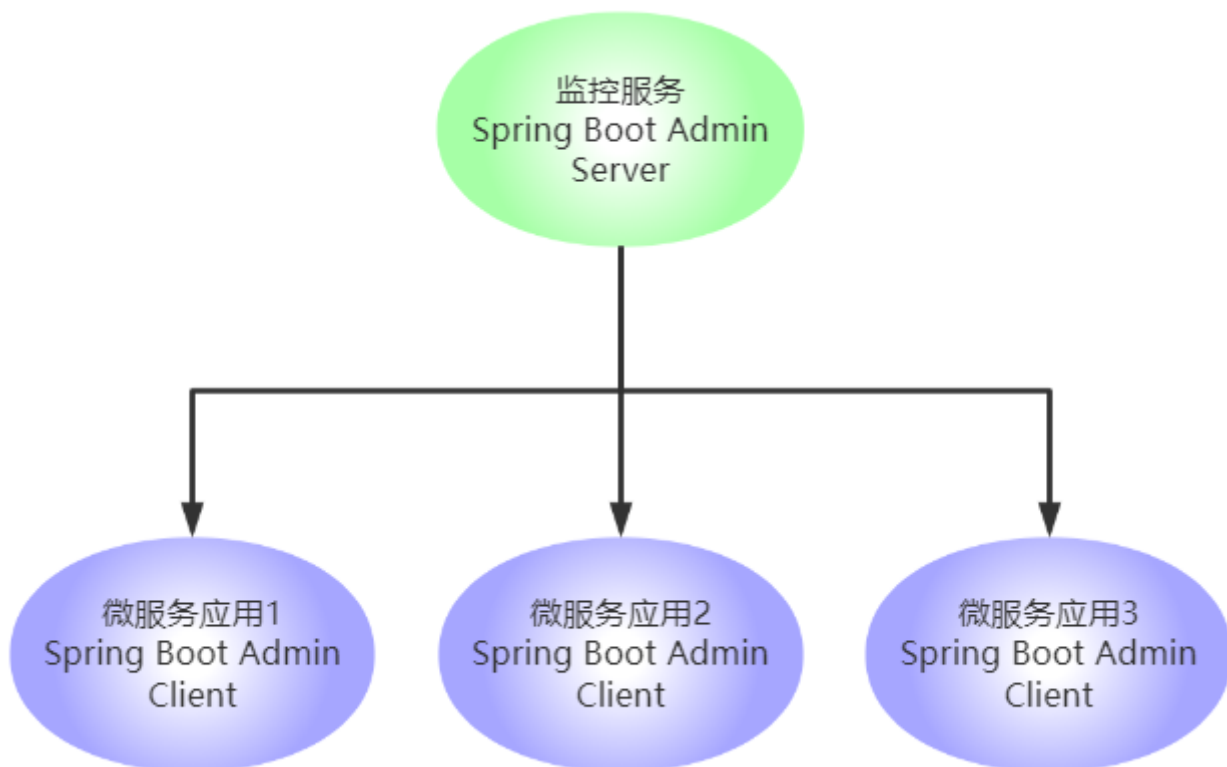
1 关于Spring Boot Admin

Spring Boot Admin是一个管理和监控Spring Boot应用程序的开源项目, 在对单一应用服务监控的同时也提供了集群监控方案, 支持通过eureka、consul、zookeeper等注册中心的方式实现多服务监控与管理。Spring Boot Admin UI部分使用Vue JS将数据展示在前端。

Spring Boot Admin分为服务端 (spring-boot-admin-server) 和客户端 (spring-boot-admin-client) 两个组件:

- spring-boot-admin-server: 通过采集actuator端点数据显示在spring-boot-admin-ui上, 已知的端点几乎都有进行采集。
- spring-boot-admin-client: 是对Actuator的封装, 提供应用系统的性能监控数据。此外, 还可以通过spring-boot-admin动态切换日志级别、导出日志、导出heapdump、监控各项性能指标等。

Spring Boot Admin服务器端负责收集各个客户的数据。各台客户端配置服务器地址, 启动后注册到服务器。服务器不停地请求客户端的信息 (通过Actuator接口)。具体架构如下图所示。



上图为Spring Boot Admin的整体架构，在每个Spring Boot应用程序上增加Spring Boot Admin Client组件。这样每个Spring Boot应用即Admin客户端，Admin服务端通过请求Admin客户端的接口收集所有的Spring Boot应用信息并进行数据呈现，从而实现Spring Boot应用监控。

2 搭建运维监控平台

2.1 搭建服务端

导入依赖

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5     https://maven.apache.org/xsd/maven-4.0.0.xsd">
6     <modelVersion>4.0.0</modelVersion>
7     <parent>
8         <groupId>com.zeroone.star</groupId>
9         <artifactId>zero-one-cloud-build</artifactId>
10        <version>2.0.0-SNAPSHOT</version>
11        <relativePath/>
12    </parent>
13    <artifactId>project-admin-server</artifactId>
14    <version>1.0.0-SNAPSHOT</version>
15    <dependencies>
16        <!-- web -->
17        <dependency>
18            <groupId>org.springframework.boot</groupId>
19            <artifactId>spring-boot-starter-web</artifactId>
20        </dependency>
21        <!-- spring boot admin server -->
22        <dependency>
23            <groupId>de.codecentric</groupId>
24            <artifactId>spring-boot-admin-starter-server</artifactId>
25        </dependency>
26    </dependencies>
27    <build>
28        <plugins>
29            <plugin>
30                <groupId>org.springframework.boot</groupId>
31                <artifactId>spring-boot-maven-plugin</artifactId>
32                <configuration>
33                    <mainClass>com.zeroone.star.projectadminserver.Application</mainClass>
34                </configuration>
35            </plugin>
36        </plugins>
37    </build>
38 </project>
```

修改项目配置文件

```

1 spring:
2   application:
3     name: PROJECT-ADMIN-SERVER
4 server:
5   port: 8080

```

修改程序入口

```

1 /**
2  * <p>
3  * 描述：程序入口
4  * </p>
5  * <p>版权：&copy;01星球</p>
6  * <p>地址：01星球总部</p>
7  * @author 阿伟学长
8  * @version 1.0.0
9  */
10 @EnableAdminServer
11 @SpringBootApplication
12 public class Application {
13     public static void main(String[] args) {
14         SpringApplication.run(Application.class, args);
15     }
16 }

```

启动服务访问测试，访问地址：<http://localhost:8080>，浏览器显示效果如下图所示：



从Admin服务端的启动界面可以看到，Applications页面会展示应用数量、实例数量和状态3个信息。这里由于没有启动客户端，因此显示出“暂无应用注册”的信息。

2.2 搭建客户端

导入依赖

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5     https://maven.apache.org/xsd/maven-4.0.0.xsd">
6   <modelVersion>4.0.0</modelVersion>
7   <parent>
8     <groupId>com.zeroone.star</groupId>

```

```

7         <artifactId>zero-one-cloud-build</artifactId>
8         <version>2.0.0-SNAPSHOT</version>
9         <relativePath/>
10    </parent>
11    <artifactId>project-admin-client</artifactId>
12    <version>1.0.0-SNAPSHOT</version>
13    <dependencies>
14        <!-- web -->
15        <dependency>
16            <groupId>org.springframework.boot</groupId>
17            <artifactId>spring-boot-starter-web</artifactId>
18        </dependency>
19        <!-- spring boot admin client -->
20        <dependency>
21            <groupId>de.codecentric</groupId>
22            <artifactId>spring-boot-admin-starter-client</artifactId>
23        </dependency>
24    </dependencies>
25    <build>
26        <plugins>
27            <plugin>
28                <groupId>org.springframework.boot</groupId>
29                <artifactId>spring-boot-maven-plugin</artifactId>
30                <configuration>
31
32            <mainClass>com.zeroone.star.projectadminserver.Application</mainClass>
33            </configuration>
34        </plugin>
35    </plugins>
36    </build>
37 </project>

```

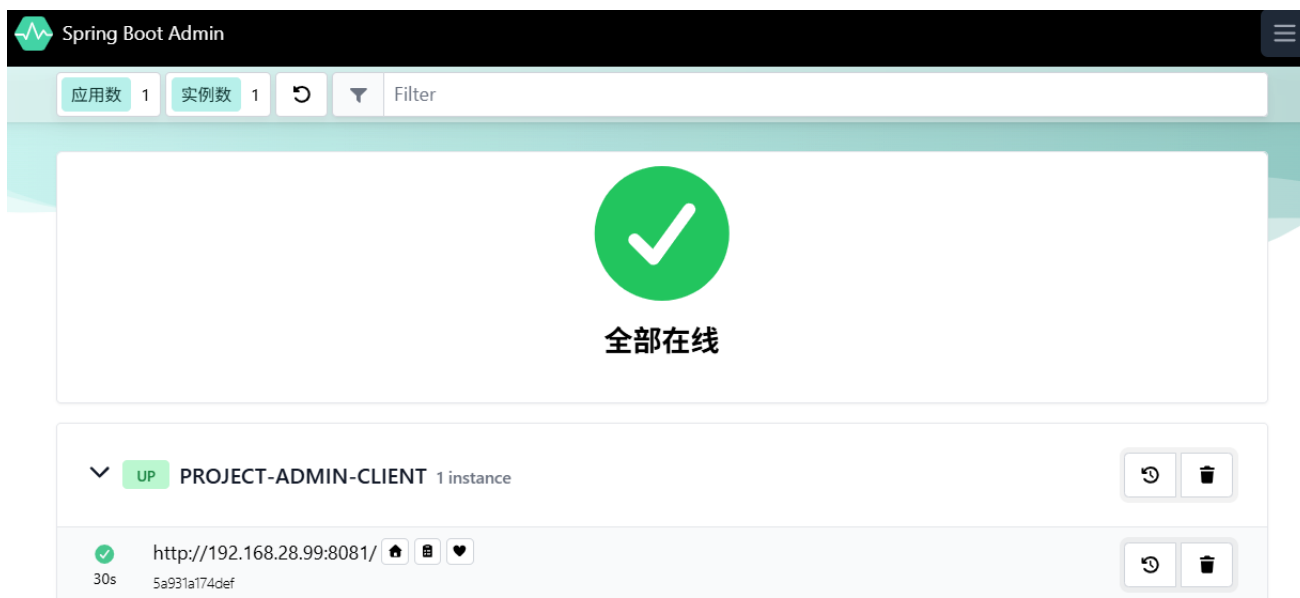
修改项目配置文件

```

1  server:
2    port: 8081
3  spring:
4    application:
5      name: PROJECT-ADMIN-CLIENT
6    boot:
7      admin:
8        client:
9          # 配置Admin服务器的地址
10         url: http://localhost:8080
11         instance:
12           # 通过IP注册
13           service-host-type: ip
14
15  # 打开客户端Actuator的监控
16  management:
17    endpoints:
18      web:
19        exposure:
20          include: "*"

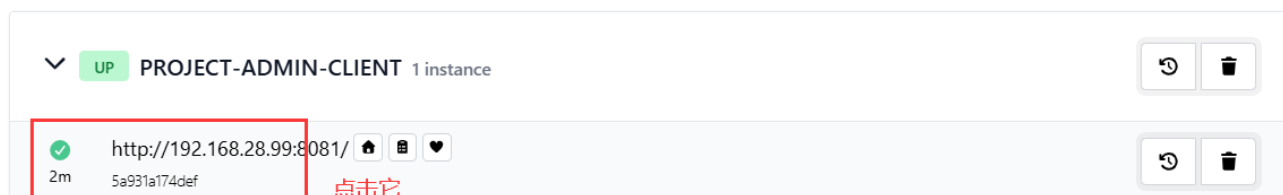
```

启动服务，然后关系监控服务端页面，可以看到如下图所示的效果



客户端启动之后，Admin服务器界面的应用数量会增加。

页面会展示被监控的应用列表，单击应用名称会进入此应用的详细监控信息页面。



可以看到下面所示的效果



这个页面会实时显示应用的运行监控信息，包括Actuator所有的端点数据信息。

Spring Boot Admin以图形化的形式展示了应用的各项信息，这些信息大多来自于Spring Boot Actuator提供的接口。利用图形化的形式很容易看到应用的各项参数变化，甚至有些页面还可以进行一些配置操作，比如改变打印日志的级别等。

3 告警提醒功能

Spring Boot Admin提供了强大的提醒功能，能够在发生服务状态变更的时候发出告警。支持的Email等提醒功能，同时也支持自定义告警提醒。

3.1 邮件提醒

在服务端添加邮件组件

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-mail</artifactId>
4 </dependency>
```

修改配置文件

```
1 spring:
2   application:
3     name: PROJECT-ADMIN-SERVER
4     # 邮件服务器配置
5   mail:
6     # 邮箱站点配置，这是使用QQ邮箱的smtp协议
7     host: smtp.qq.com
8     port: 465
9     # 登录邮箱账号
10    username: xxxxx@qq.com
11    # 授权码
12    password: xxxxxxxx
13    properties:
14      mail:
15        smtp:
16          auth: true
17          ssl:
18            enable: true
19    boot:
20      admin:
21        notify:
22          mail:
23            # 开启邮件通知
24            enabled: true
25            # 发件人邮箱地址和邮箱服务器用户名保持一致
26            from: xxxxx@qq.com
27            # 接收通知的收件人
28            to: toxxxxx@qq.com
29    server:
30      port: 8080
```

如何获取授权码，下面以QQ邮箱为例，在邮箱设置中找到SMTP服务，开启它。

参考链接: <https://service.mail.qq.com/cgi-bin/help?subtype=1&id=10000&no=1001607>

QQ邮箱

mail.qq.com

搜索

开通 VIP 会员，让邮箱更好用 >

阿伟 · 2000000000@qq.com 普通用户

1 设置

← 返回

重要联系人

代收邮箱

自助查询

应用

应用管理

账号

账号与安全 2

辅助

阿伟 普通用户

开通 VIP 会员，让邮箱更好用

立即开通
最低每月只需 17.5 元

进入会员中心

通用

个人信息

我的资料卡

阿伟

编辑

生日

公历 1月 1日

☒ QQ好友可见我的生日

快过生日时，QQ好友可以在邮箱日历或贺卡页看到你的生日。

安全设置 1

设备管理

设置独立密码后，进入邮箱需要输入独立密码验证，使用QQ邮箱更加安全

设置独立密码

文件夹区域加锁

“文件夹区域”是由“我的文件夹”、“代收邮箱”、“记事本”组成。加锁即对这几部分设置密码，以保护你的信息

设置加锁密码

POP3/IMAP/SMTP/Exchange/CardDAV 服务（未开启）

开启该服务即可在第三方邮件客户端收取QQ邮箱的邮件[配置方法](#)

开启服务 2



短信验证

请用手机，发送短信，然后点击“我已发送”进行验证



1

扫描发送短信验证码

微信扫描二维码，可快速发送短信

[手动发送短信](#)（短信费用由运营商收取）

选择其他方式验证

我已发送

2

发送后
点击



SMTP/IMAP 服务已开启

在第三方客户端登录时，密码框请输入以下授权码：

tkmpbna...

1

复制授权码

你可以拥有多个授权码，无需记住该授权码，为了安全请勿告知其他人。

备注授权码用途：spring boot admin 通知

2

填写备注

返回安全设置

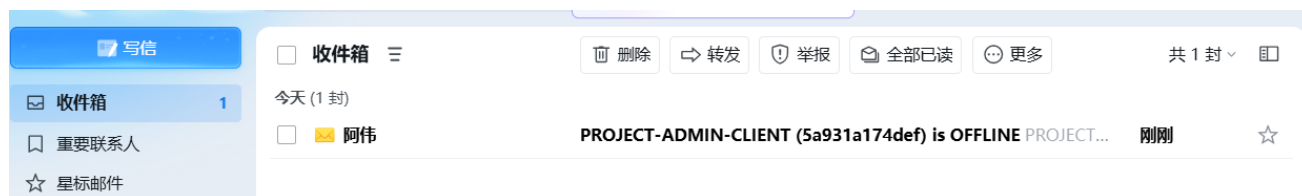
3

然后可以返回，授权码请妥善保管哦

将复制的授权码填入到application.yml中

然后测试验证，再次启动服务端和客户端，然后停止客户端，模拟应用宕机的情况。这样Spring Boot Admin 就会发送告警邮件提醒。

邮箱中收到邮件，效果如下图所示



PROJECT-ADMIN-CLIENT (5a931a174def) is OFFLINE ☆



阿伟 <[redacted]@qq.com>

收件人 我 <[redacted]@qq.com>

🌐 邮件可翻译为中文 [全文翻译](#)

PROJECT-ADMIN-CLIENT (5a931a174def) is OFFLINE

Instance 5a931a174def changed status from UP to OFFLINE

Status Details

exception

org.springframework.web.reactive.function.client.WebClientRequestException

message

Connection refused: no further information: /192.168.28.99:8081

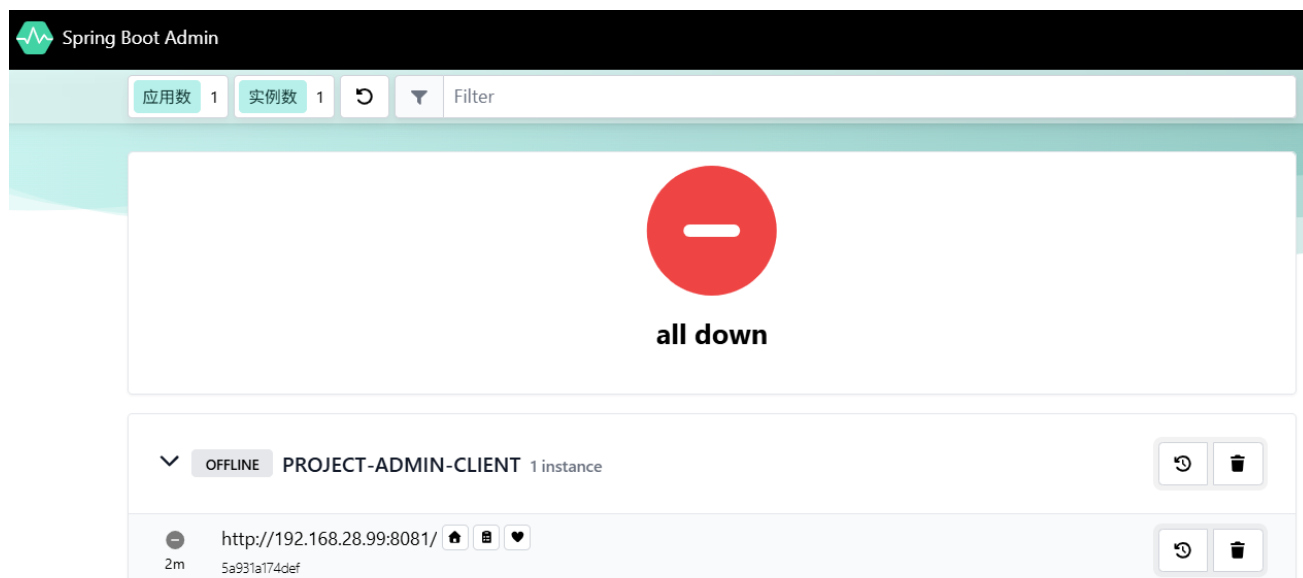
Registration

Service Url <http://192.168.28.99:8081/>

Health Url <http://192.168.28.99:8081/actuator/health>

Management Url <http://192.168.28.99:8081/actuator>

服务端控制台也会看到客户端离线



3.2 自定义告警

除了邮件提醒之外，通常我们还需要其他的提醒方式，比如：短信，日志等。我们可以通过自定义的方式实现自定义的消息告警方式。

Spring Boot Admin 实现自定义告警提醒也非常简单，只要实现Notifier接口即可。具体实现方式：继承AbstractEventNotifier或AbstractStatusChangeNotifier这两个类。然后重写doNotify中实现具体的业务逻辑。

下面通过示例演示自定义告警提醒功能：

首先，创建LoggingNotifier类，实现告警提醒功能，示例代码如下：

```
1  /**
2   * <p>
3   * 描述：自定义一个日志告警示例
4   * </p>
5   * <p>版权：&copy;01星球</p>
6   * <p>地址：01星球总部</p>
7   * @author 阿伟学长
8   * @version 1.0.0
9   */
10 @Component
11 public class LoggingNotifier extends AbstractEventNotifier {
12     private static final Logger LOGGER =
13     LoggerFactory.getLogger(LoggingNotifier.class);
14     protected LoggingNotifier(InstanceRepository repository) {
15         super(repository);
16     }
17     @Override
18     protected Mono<Void> doNotify(InstanceEvent event, Instance instance) {
19         return Mono.fromRunnable(() -> {
20             if (event instanceof InstanceStatusChangedEvent) {
21                 LOGGER.info("Instance {} ({{}) is {}",
22                     instance.getRegistration().getName(),
23                     event.getInstance(),
24                     ((InstanceStatusChangedEvent)
25 event).getStatusInfo().getStatus());
26             } else {
27                 LOGGER.info("Instance {} ({{}) {}",
28                     instance.getRegistration().getName(),
29                     event.getInstance(),
30                     event.getType());
31             }
32         });
33     }
34 }
```

然后，再次运行服务端和客户端。启动成功之后，再关掉客户端，模拟应用宕机的情况。

```
[ctor-http-nio-3] c.z.s.p.components.LoggingNotifier : Instance PROJECT-ADMIN-CLIENT (5a931a174def) is OFFLINE
```

我们看到服务端后台日志显示，服务端已经收到了客户端状态改变的告警消息。客户端状态已经变为OFFLINE。

4 使用注册中心

因为nacos自动帮我们整合了与admin的关联工作,只需要注册进nacos,并且与服务端保持在同一命名空间和分组下即可。

4.1 服务端调整

添加nacos依赖

```
1 <!-- spring cloud alibaba -->
2 <dependency>
3     <groupId>com.alibaba.cloud</groupId>
4     <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
5 </dependency>
```

修改配置文件

```
1 spring:
2     # .....
3     # nacos配置
4     cloud:
5         nacos:
6             server-addr: 192.168.28.132:8848
7             username: nacos
8             password: nacos
9
10    management:
11        endpoints:
12            web:
13                exposure:
14                    include: '*'
15        endpoint:
16            health:
17                show-details: always
```

修改程序入口

```
1 @EnableAdminServer
2 @EnableDiscoveryClient
3 @SpringBootApplication
4 public class Application {
5     public static void main(String[] args) {
6         SpringApplication.run(Application.class, args);
7     }
8 }
```

4.2 客户端调整

添加nacos相关依赖，参考4.1服务端调整

修改配置文件

```
1 server:
2     port: 8081
3 spring:
4     application:
5         name: PROJECT-ADMIN-CLIENT
```

```

6 # boot:
7 #   admin:
8 #     client:
9 #       # 配置Admin服务器的地址
10 #       url: http://localhost:8080
11 #       instance:
12 #       # 通过IP注册
13 #       service-host-type: ip
14 # nacos配置
15 cloud:
16   nacos:
17     server-addr: 192.168.28.132:8848
18     username: nacos
19     password: nacos
20
21 # 打开客户端Actuator的监控
22 management:
23   endpoints:
24     web:
25       exposure:
26         include: "*"

```

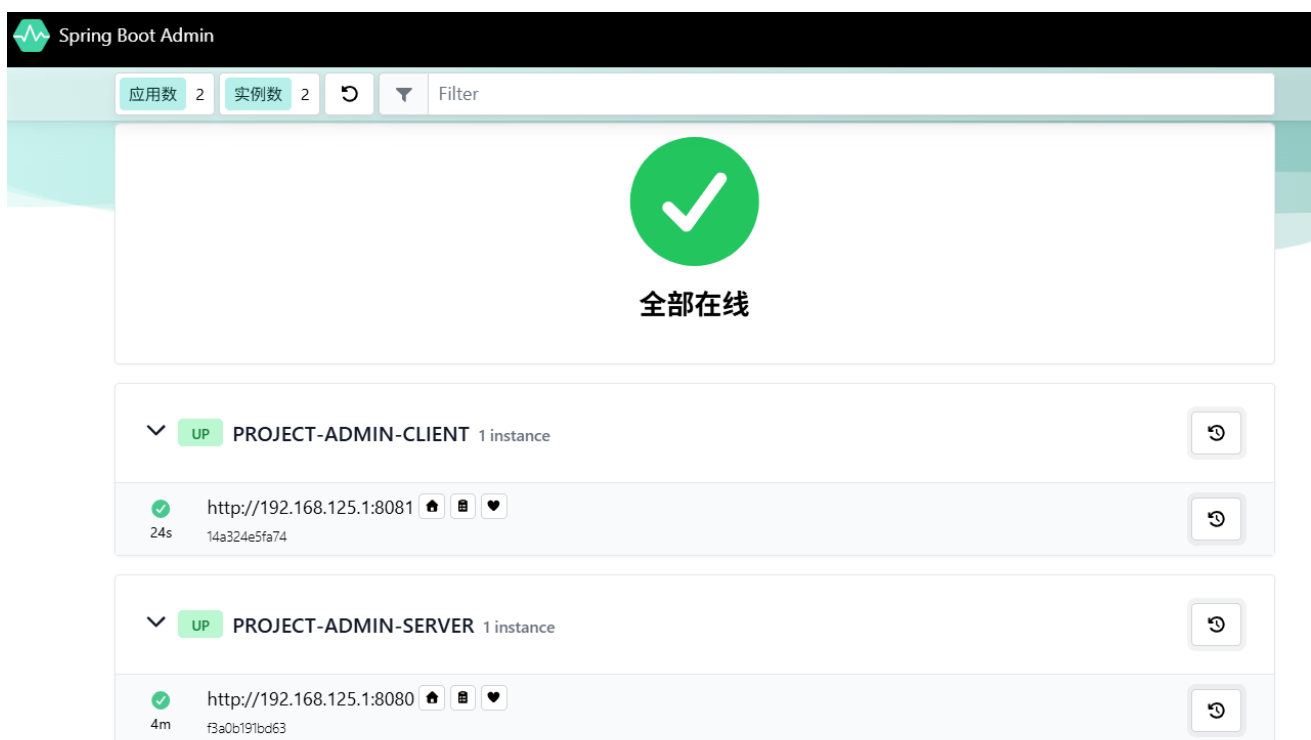
修改程序入口

```

1 @EnableDiscoveryClient
2 @SpringBootApplication
3 public class Application {
4     public static void main(String[] args) {
5         SpringApplication.run(Application.class, args);
6     }
7 }

```

启动服务端和客服端测试一下



4.3 服务端认证

下面为服务器加入安全证

添加依赖

```
1  <!-- security -->
2  <dependency>
3      <groupId>org.springframework.boot</groupId>
4      <artifactId>spring-boot-starter-security</artifactId>
5  </dependency>
```

修改配置

```
1  spring:
2      # .....
3      # nacos配置
4      cloud:
5          nacos:
6              server-addr: 192.168.28.132:8848
7              username: nacos
8              password: nacos
9              discovery:
10                 metadata:
11                     user.name: admin
12                     user.password: admin123
13
14     # 认证账号配置
15     security:
16         user:
17             name: admin
18             password: admin123
19     # .....
```

编写security配置

```
1  /**
2   * <p>
3   * 描述: Security配置
4   * </p>
5   * <p>版权: &copy;01星球</p>
6   * <p>地址: 01星球总部</p>
7   * @author 阿伟学长
8   * @version 1.0.0
9   */
10 @EnableWebSecurity
11 @Configuration
12 public class SecuritySecureConfig {
13     private final AdminServerProperties adminServer;
14
15     public SecuritySecureConfig(AdminServerProperties adminServerProperties) {
16         this.adminServer = adminServerProperties;
17     }
18 }
```

```

19     @Bean
20     protected SecurityFilterChain filterChain(HttpSecurity http) throws Exception
21     {
22         // 登录成功处理器
23         SavedRequestAwareAuthenticationSuccessHandler successHandler = new
24         SavedRequestAwareAuthenticationSuccessHandler();
25         successHandler.setTargetUrlParameter("redirectTo");
26         successHandler.setDefaultTargetUrl(this.adminServer.path("/"));
27         http
28             // 请求授权配置
29             .authorizeHttpRequests(auth -> auth
30                 // 静态资源和登录页允许访问
31                 .requestMatchers("/css/**", "/js/**", "/image/**",
32                     adminServer.path("/assets/**"),
33                     adminServer.path("/login")).permitAll()
34                 // 其他所有请求都需要认证
35                 .anyRequest().authenticated()
36             )
37             // 表单登录
38             .formLogin(form -> form
39                 .loginPage(adminServer.path("/login"))
40                 .successHandler(successHandler)
41             )
42             // 注销配置
43             .logout(logout -> logout
44                 .logoutUrl(adminServer.path("/logout"))
45                 // 注销后跳回登录页
46                 .logoutSuccessUrl(adminServer.path("/login?logout"))
47                 .invalidateHttpSession(true)
48                 .clearAuthentication(true)
49                 .permitAll()
50             )
51             // 启用Basic认证
52             .httpBasic(Customizer.withDefaults())
53             // CSRF配置
54             .csrf(csrf -> csrf
55                 .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
56                 .ignoringRequestMatchers(
57                     adminServer.path("/instances"),
58                     adminServer.path("/actuator/**"),
59                     adminServer.path("/logout"))
60             );
61     }

```

启动服务后，就会出现登录界面



Spring Boot Admin

用户名

密码

☐ 记住用户

登录

输入账号和密码就可以进入到应用页面